

Print a message in UART console

Name: Praveen
CDSID: PKASIRAJ



Table of Contents

1. Introduction	3
2. Scope	3
3. Prerequisite	3
3.1.Hardware	3
3.2.Software	3
4. Connection Diagram	3
5. Working Principle	4
6. Application code	5
7. Hardware Image	6
8. Appendix	6

1.Introduction:

In this experiment, we have used the UART communication protocol with the Raspberry Pi Pico microcontroller to send a message to a PC's serial console. UART is a common way for devices to talk to each other using just two wires. The Raspberry Pi Pico is a powerful and efficient microcontroller for this because it's simple and affordable. The software and hardware are used and then a C program is written that sends the message "Hello Riser" to a serial console on a PC. This will help us understand how UART communication works.

2.Scope:

This experiment involves setting up a Raspberry Pi Pico microcontroller and a host PC for UART communication. You'll write a C program to send a message via UART and check the output using a serial console application (PuTTY). We'll understand the necessary prerequisites, connection details, and step-by-step instructions. Learning the basics of UART communication, its importance, and how it's used in microcontroller projects. By the end, getting to know how to configure UART settings, write and compile a basic C program for UART communication, and troubleshoot common issues during setup and execution.

3.Prerequisite:

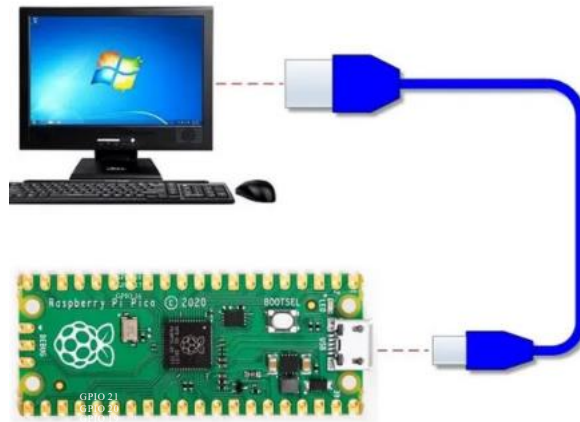
3.1. Hardware:

- **Raspberry Pi Pico:** A budget-friendly, high-efficiency microcontroller board featuring a dual-core Arm Cortex-M0+ processor.
- **Personal Computer (PC):** To use the serial console application and connect with the Raspberry Pi Pico.
- **USB Power Cable:** A micro-USB cable that powers the Pico and enables USB to Serial communication.

3.2. Software:

- **PuTTY:** A free, open-source terminal emulator supporting various network protocols, including serial communication. It will be used to monitor the UART output from the Pico.
- **Pico SDK:** A comprehensive software development kit for the Raspberry Pi Pico, offering libraries and tools for C/C++ programming.
- **GCC (GNU Compiler Collection):** A compiler system supporting various programming languages, essential for compiling the C program.
- **Device Driver:** Ensure the USB to Serial driver is installed on the PC to recognize the Pico as a serial device.

4.Connection Diagram:



5.Working Principle:

The process involves setting up both hardware and software to enable UART communication between the Raspberry Pi Pico and the PC. First, Pico is powered and recognized by the PC as a serial device. Then, the serial console application (Putty) is configured to communicate with Pico using the correct COM port and a baud rate of 115200. A C program is written and compiled with the Pico SDK and GCC, which initializes the UART interface on the Pico and sends a predefined message. This message is transmitted via UART to the PC and displayed on the Putty console. The program runs in a continuous loop to keep the message visible and the UART interface active.

6. Application code:

Riser Lab Experiment

```
hello_riser.c > main()
#include <stdio.h>
#include "pico/stdlib.h"

int main(){
    //Initialize I/O
    stdio_init_all();

    //Initiaze GPIO
    gpio_init(25);
    gpio_set_dir(25, GPIO_OUT);

    //Infinite Loop
    while(1)
    {
        gpio_put(25, 1);
        printf("Hello Riser\n");
        sleep_ms(500);
    }
}
```

CmakeLists.txt

```
cmake_minimum_required(VERSION 3.21)
```

```
# Include the Pico SDK
include(pico_sdk_import.cmake)
```

```
project(pico-projects)
```

```
pico_sdk_init()
```

```
# Add an executable target named 'hello_riser' with the source file 'hello_riser.c'
add_executable(hello_riser
    hello_riser.c
)
```

```
# Link the Pico standard library to the 'hello_riser' target
target_link_libraries(hello_riser pico_stdlib)
```

```
# Enable USB standard input/output for the 'hello_riser' target
pico_enable_stdio_usb(hello_riser 1)
```

```
# Disable UART standard input/output for the 'hello_riser' target
pico_enable_stdio_uart(hello_riser 0)
```

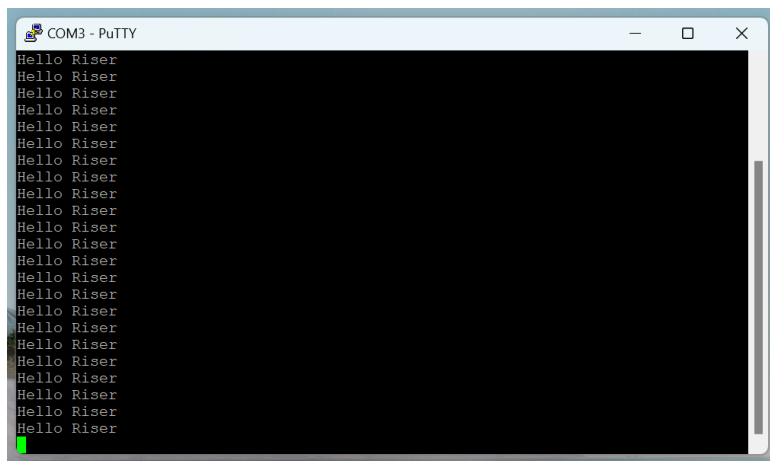
```
# Add extra output formats (like UF2) for the 'hello_riser' target for flashing  
pico_add_extra_outputs(hello_riser)
```

7. Output Image:

- **Hardware:**



- **Software:**



8. Appendix:

- **Baud Rate:** This is the speed at which data is transferred in a communication channel. For this experiment, it's set to 115200 bits per second.
- **COM Port:** This is the interface through which the PC communicates with Pico. There is a need to select the correct COM port in PuTTY.
- **PuTTY Configuration:** Make sure the correct COM port and baud rate are selected in PuTTY for successful communication. Other settings like data bits, stop bits, and parity should also match Pico's configuration.